

Event Management System

Implementation Documentation

Version 1.0 | Full-Stack Technical Specification

1. System Overview

The Event Management System (EMS) is a web application that enables colleges to organize, approve, and track student participation in both on-campus (In-College) and off-campus (Out-College) events. The platform serves three user roles: Student, Teacher (Coordinator), and Admin.

1.1 Core Capabilities

- Secure role-based login and sign-up with college email verification
- Admin approval workflow for new Teacher accounts
- Event lifecycle: creation → approval → participation → attendance
- Student-initiated Out-College event requests with brochure and proof-of-attendance uploads
- Teacher tools: review applications, mark attendance, add approval comments
- Color-coded dashboard for each role with participation counters

2. Technology Stack

Layer	Technology	Purpose
Frontend	React 18 + Vite	SPA with role-based routing
Styling	Tailwind CSS + shadcn/ui	Component library and utility styling
State Mgmt	Zustand	Global auth/session store
Backend	Node.js + Express.js	REST API server
Database	PostgreSQL 16	Relational data store
ORM	Prisma	Type-safe database access
Auth	JWT + bcrypt	Stateless auth, password hashing
File Storage	AWS S3 / local disk	Brochures, proof-of-attendance uploads
Email	Nodemailer + SMTP	OTP / notification emails
Validation	Zod (BE) + React Hook Form (FE)	Schema validation on both ends

3. Database Schema

All entities and relationships are managed by Prisma ORM with PostgreSQL. The schema below describes every table and its key columns.

3.1 Users Table

Column	Type	Notes
id	UUID (PK)	Auto-generated
name	VARCHAR(120)	Full name
email	VARCHAR(180) UNIQUE	Must be college domain email
password_hash	TEXT	bcrypt hashed
role	ENUM	'student' 'teacher' 'admin'
status	ENUM	'pending' 'active' 'rejected' — Teacher accounts start pending
created_at	TIMESTAMP	Auto
updated_at	TIMESTAMP	Auto

3.2 Events Table

Column	Type	Notes
id	UUID (PK)	
title	VARCHAR(200)	
description	TEXT	
type	ENUM	'in_college' 'out_college'
category	ENUM	'volunteer' 'participant' 'both'
start_date	DATE	
end_date	DATE	
status	ENUM	'draft' 'pending' 'approved' 'rejected'
created_by	UUID (FK -> users)	Student or Admin who created it
coordinator_id	UUID (FK -> users)	Assigned teacher
brochure_url	TEXT	Out-College events only
admin_comment	TEXT	Approval/rejection note

3.3 Registrations Table

Column	Type	Notes
id	UUID (PK)	
event_id	UUID (FK -> events)	
student_id	UUID (FK -> users)	
role_type	ENUM	'volunteer' 'participant'
status	ENUM	'pending' 'accepted' 'rejected'
proof_url	TEXT	Out-College proof upload
registered_at	TIMESTAMP	

3.4 Attendance Table

Column	Type	Notes
id	UUID (PK)	
event_id	UUID (FK -> events)	
student_id	UUID (FK -> users)	
date	DATE	Day of attendance (e.g. Day 1 = event start_date)
present	BOOLEAN	Default false
marked_by	UUID (FK -> users)	Coordinator who marked it

4. Backend Implementation

4.1 Project Structure

```
server/
  src/
    config/          # DB, env, AWS config
    middleware/      # auth guard, role guard, error handler
    modules/
      auth/          # login, signup, OTP
      users/         # profile, admin approval
      events/        # CRUD, approval, assign coordinator
      registrations/ # join, accept, reject
      attendance/    # mark per day, fetch records
      uploads/       # S3 signed URLs
    prisma/          # schema.prisma, migrations
    app.ts           # Express setup
    server.ts        # listen
```

4.2 Authentication & Sign-Up Flow

Student Sign-Up

- POST /api/auth/signup { name, email, password, role: 'student' }
- Validate email against allowed college domain suffix (e.g. @college.edu)
- Hash password with bcrypt (rounds = 12)
- Set status = 'active' immediately for students
- Return JWT (1 day) on success

Teacher Sign-Up

- POST /api/auth/signup { name, email, password, role: 'teacher' }
- Same college email validation
- Set status = 'pending' — account is inactive until Admin approves
- Send email notification to Admin(s) about pending approval

Login

- POST /api/auth/login { email, password }
- Check status: if 'pending', return 403 with message 'Account awaiting admin approval'
- Verify bcrypt hash
- Sign JWT payload: { userId, role, name } — expires in 1 day
- Return token + user object (no password)

4.3 Middleware

Middleware	Purpose
authGuard	Verifies JWT from Authorization: Bearer <token> header; attaches req.user
roleGuard(roles[])	Checks req.user.role is in the allowed list; 403 otherwise
uploadMiddleware	Multer config for file uploads (brochure, proof); 10 MB limit, PDF/JPG/PNG only
errorHandler	Central Express error handler — returns consistent { error, message } JSON

4.4 API Endpoints

4.4.1 Auth Routes — /api/auth

Method	Path	Access	Description
POST	/signup	Public	Register student or teacher
POST	/login	Public	Login; returns JWT
GET	/me	Authenticated	Fetch logged-in user profile

4.4.2 User / Admin Routes — /api/users

Method	Path	Access	Description
GET	/pending-teachers	Admin	List teachers with status = pending
PATCH	/:id/approve	Admin	Approve teacher; set status = active
PATCH	/:id/reject	Admin	Reject teacher account
GET	/teachers	Admin	List all active teachers (for coordinator assignment)

4.4.3 Event Routes — /api/events

Method	Path	Access	Description
GET	/	Authenticated	List events (filtered by role & status)
POST	/	Admin Student	Create event (Admin = in-college; Student = out-college)
GET	/:id	Authenticated	Get single event details
PATCH	/:id/approve	Admin Teacher	Approve event; optionally add comment
PATCH	/:id/reject	Admin Teacher	Reject with comment
PATCH	/:id/assign	Admin	Assign coordinator (teacher) to event
POST	/:id/brochure	Student	Upload brochure for out-college event

4.4.4 Registration Routes — /api/registrations

Method	Path	Access	Description
POST	/	Student	Join event as volunteer or participant
GET	/my	Student	List student's own registrations + status
GET	/event/:eventId	Teacher Admin	List all registrations for an event
PATCH	/:id/accept	Teacher Admin	Accept student registration
PATCH	/:id/reject	Teacher Admin	Reject student registration
POST	/:id/proof	Student	Upload proof of attendance (Out-College)

4.4.5 Attendance Routes — /api/attendance

Method	Path	Access	Description
POST	/mark	Teacher	Mark attendance for one student on a specific date
POST	/bulk	Teacher	Bulk mark attendance for all students on a date
GET	/event/:eventId	Teacher Admin	Fetch full attendance sheet for an event
GET	/student/:studentId	Admin Student (own)	Fetch all attendance records for a student

5. Frontend Implementation

5.1 Project Structure

```
client/  
  src/  
    api/           # Axios instance + API call functions  
    assets/        # Logo, icons  
    components/    # Shared UI components  
      common/      # Button, Input, Badge, Modal, FileUpload  
      layout/      # Navbar, Sidebar, BottomNav  
    pages/  
      auth/        # Login.tsx, SignUp.tsx  
      student/     # Dashboard, Events, Participation, Volunteering  
      teacher/     # Dashboard, Events, EventDetail  
      admin/       # Dashboard, Events, AdminPanel  
    store/         # Zustand auth store  
    hooks/         # useAuth, useRole  
    router/        # Protected route wrappers  
    utils/         # date helpers, file utils
```

5.2 Authentication Pages

Login Page

- Clean two-column layout: left = illustrated graphic / college branding; right = form
- Email + Password fields with validation (Zod schema)
- Role indicator auto-detected from email or a toggle switch (Student / Teacher)
- On 403 (pending): show yellow banner 'Your account is pending Admin approval'
- On success: redirect to role-specific dashboard via React Router

Sign-Up Page

- Fields: Full Name, College Email, Password, Confirm Password, Role (Student / Teacher)
- Live email domain validation against allowed college suffix
- Teacher sign-up shows note: 'Your account will be active after Admin approval'
- Submit → API call → on success show success toast, redirect to Login

5.3 Student Portal

Navigation

- Bottom navigation bar (mobile) / left sidebar (desktop) with four tabs:
 - Dashboard, Events, Participation, Volunteering

Dashboard

- Welcome header with student name and college logo
- Summary cards: Active Events Joined, Pending Approvals, Volunteering Count, Participation Count

- Upcoming events strip (horizontal scroll) with color-coded status chips

Events Tab

- Segmented control: Browse Events | My Requests
- Browse: card grid of approved In-College events; each card shows title, date, type badge, slots
- 'Join' button opens a bottom sheet: select role (Volunteer / Participant), confirm
- '+ Raise Out-College Event' FAB (floating action button):
 - Form: event title, organizer, location, start/end date, description
 - Upload brochure (PDF/image) with progress bar
 - On submit: status = pending; moves to My Requests list

Participation Tab

- List of all events where student registered as Participant
- Status badge per row: Pending (yellow), Accepted (green), Rejected (red)
- For accepted Out-College events: 'Upload Proof' button (geotag photo or certificate)

Volunteering Tab

- Same layout as Participation but filtered to Volunteer registrations

5.4 Teacher Portal

Navigation

- Left sidebar (desktop) / bottom tabs (mobile): Dashboard | Events

Dashboard

- Stats: Events Assigned, Pending Student Approvals, Attendance Marked Today
- Events table listing all events the teacher is coordinator of, with status and counts

Events Tab

- List of assigned events with search and date-range filter
- Click event → Event Detail page:
 - Event info header (title, dates, type, status)
 - Tabs inside: Registrations | Attendance | Out-College Proof Review
- Registrations sub-tab:
 - Table: student name, email, role type, status, action buttons (Accept / Reject)
 - Bulk accept / reject with checkbox selection
- Attendance sub-tab:
 - Date picker showing each day of the event (Day 1, Day 2 ...)
 - Student grid with toggle per student for that day (Present / Absent)
 - 'Save Attendance' button — POST /api/attendance/bulk
- Out-College Proof sub-tab (only for Out-College events):

- Table of students who uploaded proof; preview thumbnail link; Accept/Reject buttons
- Teacher can add a short comment on rejection

5.5 Admin Portal

Navigation

- Sidebar: Dashboard | Events | Admin Panel

Dashboard

- Platform stats: Total Events, Active Students, Pending Approvals, Teachers
- Color-coded calendar widget showing event dates (in-college = blue, out-college = teal, pending = amber)

Events Tab

- Full event table with filters: All / In-College / Out-College / Pending / Approved / Rejected
- '+ Create Event' button opens event creation form (In-College only for Admins):
 - Title, description, category (Volunteer/Participant/Both), dates
 - Assign Coordinator dropdown (populated from active teachers list)
- Approve/Reject In-College events submitted by teachers or by the system
- Approve/Reject Out-College events raised by students
- 'Add Comment' field on approval/rejection (persisted in events.admin_comment)

Admin Panel Tab

- Pending Teacher Accounts section: list with name, email, registered date
- Approve / Reject buttons per row — calls PATCH /api/users/:id/approve (or reject)
- On approve: teacher status set to active, email notification sent to teacher

6. Authentication & Security

Concern	Implementation
Password storage	bcrypt with 12 salt rounds — plain-text passwords never stored
Token	JWT signed with HS256; payload: { userId, role }; expiry: 24 h
Token storage (FE)	localStorage (or httpOnly cookie if higher security is needed)
Protected routes	React Router <ProtectedRoute> checks auth store; redirects unauthenticated users
Role enforcement	Backend roleGuard middleware on every sensitive endpoint
Email domain check	Regex on signup: /@yourdomain\.edu\$/i — configurable via env var
File upload safety	Multer: whitelist MIME types, max 10 MB, random UUID filename
Rate limiting	express-rate-limit on /api/auth/* — max 20 req / 15 min per IP
CORS	Configured to allow only the frontend origin via env var

7. File Upload Strategy

7.1 Accepted Upload Types

Upload	Actor	Allowed Formats	Max Size
Event Brochure	Student	PDF, JPG, PNG	10 MB
Proof of Attendance	Student	JPG, PNG (geotagged), PDF	10 MB

7.2 Storage Flow

- On upload: Multer saves file to temp disk buffer
- Backend reads buffer and streams to S3 bucket (or local /uploads directory in dev)
- Returns a signed URL or public URL stored in the DB column (brochure_url / proof_url)
- Frontend displays preview using the returned URL (PDF iframe, image thumbnail)

8. Environment Configuration

Backend (.env)

```

DATABASE_URL=postgresql://user:pass@localhost:5432/ems
JWT_SECRET=your_secret_key_here
COLLEGE_EMAIL_DOMAIN=@college.edu

```

```
AWS_ACCESS_KEY_ID=...  
AWS_SECRET_ACCESS_KEY=...  
AWS_S3_BUCKET=ems-uploads  
SMTP_HOST=smtp.gmail.com  
SMTP_PORT=587  
SMTP_USER=noreply@college.edu  
SMTP_PASS=...  
PORT=5000
```

Frontend (.env)

```
VITE_API_BASE_URL=http://localhost:5000/api
```

9. Complete Event Lifecycle Summary

9.1 In-College Event (Admin-created)

Step	Actor	Action	Status Change
1	Admin	Creates event via Events tab form	draft → pending
2	Admin	Approves own event (or another admin does); assigns coordinator	pending → approved
3	Student	Browses Events tab; clicks Join; selects Volunteer or Participant	Registration: pending
4	Teacher	Opens Registrations sub-tab; clicks Accept	Registration: accepted
5	Teacher	Opens Attendance sub-tab; selects date; marks present/absent	Attendance record created

9.2 Out-College Event (Student-raised)

Step	Actor	Action	Status Change
1	Student	Raises Out-College event via Events tab; uploads brochure	Event: pending
2	Admin / Teacher	Reviews request in Events tab; approves with optional comment	Event: approved
3	Student	Attends the external event	
4	Student	Uploads proof (geotagged photo or certificate) in Participation tab	Registration: proof uploaded
5	Admin / Teacher	Reviews proof in Out-College Proof sub-tab; accepts or rejects with comment	Registration: accepted / rejected

10. Dashboard Color Coding

Color	Meaning
Blue (#2E6DA4)	Approved In-College event
Teal (#1A8A7A)	Approved Out-College event
Amber (#F59E0B)	Pending approval (any event)
Red (#DC2626)	Rejected event or registration
Green (#16A34A)	Accepted / attended
Grey (#6B7280)	Draft / inactive

All status badges in the student portal follow the same color palette so users can scan their registration statuses at a glance without reading the text.

11. Development & Deployment Notes

- Run database migrations: `npx prisma migrate dev`
- Seed admin user: `npx ts-node prisma/seed.ts`
- Start dev server: `npm run dev` (concurrently starts Express + Vite)
- Production build: `npm run build` in `client/`, serve `dist/` via Express static or Nginx
- Docker Compose provided for PostgreSQL and pgAdmin in development
- CI/CD: GitHub Actions workflow for lint, test (Vitest + Supertest), and deploy